

Digital Design

Complete Course Summary

Number Systems • Boolean Algebra • Combinational Logic • Sequential Logic • HDL / Verilog •
Memory

*How to use this summary: chapters follow the standard Digital Design textbook order (Mano & Ciletti).
Each chapter opens with theory and key formulas, followed by circuit diagrams for visual reinforcement.
The Quick Reference at the end collects every formula on one page.*

Table of Contents

Note: Page numbers shown as “1” are placeholders — open the document in Word and select “Update Table” to generate correct page numbers.

Table of Contents	2
Chapter 1 — Number Systems & Codes.....	6
1.1 Base-r Number Systems	6
1.2 Memory Units	6
1.3 Number-Base Conversions	6
1.4 Complements	6
r's Complement (2's complement for binary)	6
(r-1)'s Complement (1's complement for binary).....	6
1.5 Subtraction Using Complements $M - N$ in base r	6
1.6 Signed Binary Numbers.....	7
Worked Example — same decimal value in all three representations	7
Overflow Detection (2's complement arithmetic)	7
1.7 BCD and Other Codes.....	7
BCD Addition Rules	7
Parity Codes — Error Detection.....	8
Chapter 2 — Boolean Algebra	8
2.1 Huntington Postulates	8
2.2 Key Theorems.....	8
Duality Principle	9
2.3 Operator Precedence	9
2.4 Positive and Negative Logic	9
2.5 Logic Gate Symbols & Truth Tables	9
2.6 Canonical Forms & Standard Forms.....	10
2.7 The 16 Boolean Functions of Two Variables	11
Chapter 3 — Gate-Level Minimization (Karnaugh Maps)	11
3.1 K-Map Rules	11
4-Variable K-Map Layout (variable order)	12
3.2 SOP Minimization (group the 1s).....	12
3.3 POS Minimization (group the 0s).....	12
3.4 Prime Implicants & Don't-Cares.....	13
3.5 NAND / NOR Implementation	13
NAND — SOP procedure	13
NOR — POS procedure (dual of NAND).....	13
3.6 XOR / XNOR Functions.....	13
3.7 Combinational Hazards (Glitches) 13	
3.8 Quine-McCluskey (Tabular) Method 14	

Chapter 4 — Combinational Logic Circuits.....	14
4.1 Design & Analysis Procedures	14
Design Procedure	14
Analysis Procedure.....	14
4.2 Half Adder	15
4.3 Full Adder	15
4.4 Binary Adder & Carry Look-Ahead	15
Carry Look-Ahead Equations	16
4.5 Binary Adder-Subtractor	16
4.6 BCD Adder	16
4.7 Binary Multiplier.....	16
4.8 Magnitude Comparator.....	17
4.9 Decoders & Encoders.....	17
Decoder.....	17
Encoder	17
Octal-to-Binary Encoder (8-to-3) equations.....	17
Priority Encoder	17
4.10 Multiplexers (MUX).....	18
4.11 Three-State Gates.....	18
Chapter 5 — Verilog HDL	18
5.1 Modeling Styles.....	18
5.2 Module Structure & Net Types.....	19
5.3 Gate Delays & Timescale	19
5.4 Vectors (Buses).....	19
5.5 Dataflow Modeling (assign)	19
5.6 Behavioral Modeling — always block	19
Blocking vs Non-Blocking	20
5.7 Loop Statements	20
5.8 Conditional Statements	20
5.9 User-Defined Primitives (UDP).....	21
5.10 Test Bench & Display Tasks.....	21
Format Specifiers for \$display / \$monitor / \$write.....	22
5.11 Verilog Operator Precedence (High → Low).....	22
Chapter 6 — Sequential Logic & Storage Elements.....	23
6.1 SR Latch (NOR-gate)	23
NAND SR Latch (active-low inputs SR)	23
6.2 SR Latch with Control Input (Gated SR)	23
6.3 D Latch (Gated).....	24
6.4 Edge-Triggered Flip-Flops.....	24

Characteristic Tables & Equations	24
6.5 Master-Slave D Flip-Flop	25
6.6 Asynchronous vs Synchronous Reset	25
6.7 Setup & Hold Times — Timing Constraints 25	
Chapter 7 — Sequential Circuit Analysis & Design	26
7.1 Analysis Procedure	26
State Table Template	26
7.2 Mealy vs Moore Machines	26
7.3 Design Procedure (7 Steps)	27
7.4 Excitation Tables	27
JK Flip-Flop	27
T Flip-Flop	28
7.5 State Reduction & Assignment	28
7.6 Self-Correcting Machines	29
Chapter 8 — Registers & Counters	29
8.1 Registers	29
8.2 Shift Registers & Serial Transfer	29
Universal Shift Register — 4 operating modes	30
8.3 Serial Adder	30
Serial Adder State Table	30
8.4 Counters	30
Ripple (Asynchronous) Binary Counter	31
Synchronous Binary Counter	31
8.5 BCD Ripple Counter — State Sequence	31
8.6 Special Counters	31
8.7 Counter with Parallel Load	32
Chapter 9 — Verilog HDL for Sequential Circuits	32
9.1 initial vs always — Behavioral Modeling Patterns	32
Clock Generation with initial + always	32
Single-Pass Stimulus (initial only)	32
9.2 HDL Models for Storage Elements	32
D Latch	32
D Flip-Flop — positive edge	32
D FF with Async Reset	33
JK Flip-Flop	33
T Flip-Flop	33
9.3 Clock Correctness Rules	33
9.4 Nonblocking Assignments & Inter-Assignment Delay	33
9.5 parameter Declaration — State Encoding	33

9.6 FSM Coding — Two-Always Style	33
Mealy Machine HDL Pattern	33
Moore Machine HDL Pattern	34
9.7 RTL Design Methodology	34
9.8 HDL Design Flow	34
Chapter 10 — Memory & Programmable Logic.....	35
10.1 RAM — Structure & Operation	35
Write & Read Procedures	35
Write:.....	36
Read:.....	36
10.2 Memory Cell (SRAM).....	36
10.3 Coincident Decoding (2D Decoding).....	36
10.4 DRAM & Address Multiplexing.....	36
10.5 ROM & Programmable Devices.....	37
32×8 ROM — Implementation	37
PLD Comparison	37
PLA Programming Convention.....	37
10.6 FPGA Architecture — LUTs & CLBs	37
Quick Reference — Key Formulas & Facts.....	38

Chapter 1 — Number Systems & Codes

1.1 Base-r Number Systems

A general base-r system represents a number as:

$$N = a_n \cdot r^n + a_{n-1} \cdot r^{n-1} + \dots + a_1 \cdot r^1 + a_0 \cdot r^0 + a_{-1} \cdot r^{-1} + \dots$$

Base	Details
Binary (base 2)	Digits: 0, 1
Octal (base 8)	Digits: 0–7 1 octal digit = 3 binary digits
Decimal (base 10)	Digits: 0–9
Hexadecimal (base 16)	Digits: 0–9, A–F 1 hex digit = 4 binary digits

1.2 Memory Units

Prefix	Value
Kilo	$2^{10} = 1,024$
Mega	2^{20}
Giga	2^{30}
Tera	2^{40}

1.3 Number-Base Conversions

- ◆ Decimal → Binary: repeated division by 2 (integer part) / repeated multiplication by 2 (fraction part)
- ◆ Octal → Binary: replace each octal digit with exactly 3 binary digits
- ◆ Hexadecimal → Binary: replace each hex digit with exactly 4 binary digits

1.4 Complements

r's Complement (2's complement for binary)

$$r^n \text{ complement of } N = r^n - N$$

Shortcut: For binary — invert all bits, then add 1

(r-1)'s Complement (1's complement for binary)

$$(r^n - 1) - N \rightarrow \text{for binary: invert all bits}$$

1.5 Subtraction Using Complements $M - N$ in base r

1. Add $M + (r\text{'s complement of } N)$
2. If $M \geq N \rightarrow$ carry out is produced; discard it — result is positive
3. If $M < N \rightarrow$ no carry; take r's complement of result — answer is negative

Example: $7 - 3$ in binary: $0111 + 1101 = 1|0100 \rightarrow$ discard carry $\rightarrow 0100 = 4 \checkmark$

1.6 Signed Binary Numbers

Representation	Description
Sign-Magnitude	MSB = sign (0=+, 1=-); remaining bits = magnitude
1's Complement	Negate: invert all bits
2's Complement	Negate: invert all bits + 1 (most widely used)

Worked Example — same decimal value in all three representations

Decimal	Sign-Magnitude	1's Complement	2's Complement
+4	0 100	0 100	0 100
+2	0 010	0 010	0 010
+1	0 001	0 001	0 001
+0	0 000	0 000	0 000
-0	1 000	1 111	— (no -0)
-1	1 001	1 110	1 111
-2	1 010	1 101	1 110
-3	1 011	1 100	1 101
-4	1 100	1 011	1 100

Overflow Detection (2's complement arithmetic)

- ◆ $(+V) + (+V)$ produces $-V \rightarrow$ overflow
- ◆ $(-V) + (-V)$ produces $+V \rightarrow$ overflow
- ◆ Arithmetic subtraction rule: $(\pm A) - (\pm B) = (\pm A) + (\mp B)$

1.7 BCD and Other Codes

Code	Description
BCD (8421)	Natural binary encoding of decimal digits 0–9
2421 Code	Weighted (2,4,2,1), self-complementing
Excess-3	BCD + 3; self-complementing; unweighted
Gray Code	Only one bit changes between consecutive values; used in K-maps and encoders
ASCII	7-bit; 94 graphic + 34 non-printing characters

BCD Addition Rules

- ◆ If result ≤ 9 and no carry $\rightarrow$ valid BCD; no correction
- ◆ If result > 9 OR carry out $\rightarrow$ add 6 (0110) to correct the digit

- ◆ Propagate any carry to the next BCD digit position

Worked Example — 5 + 6 = 11:

```

  0101   (5)
+ 0110   (6)
-----
  1011   (11 — invalid BCD, >9) → add correction
+ 0110   (add 6 to correct)
-----
0001 0001 → valid BCD for 11 (digit 1, digit 1) ✓

```

Parity Codes — Error Detection

- ◆ Even parity: total 1s (including parity bit) must be even
- ◆ Odd parity: total 1s must be odd
- ◆ Parity bit generated by XOR chain of all data bits

Example: ASCII 'A' = 100 0001. Even parity bit = 0 → 0100 0001. Odd parity bit = 1 → 1100 0001

★ Chapter 1 Key Points

Base conversion	Octal ↔ 3 bits/digit; Hex ↔ 4 bits/digit
2's complement	Invert + 1; range: -2^{n-1} to $+2^{n-1}-1$
BCD correction	Add 6 if sum > 9 or carry out
Parity	XOR of all data bits gives even parity bit

Chapter 2 — Boolean Algebra

2.1 Huntington Postulates

4. Closure: B is closed under + and ·
5. Identity: 0 is identity for +; 1 is identity for ·
6. Commutative: $x+y = y+x$ and $x \cdot y = y \cdot x$
7. Distributive: $x \cdot (y+z) = (x \cdot y) + (x \cdot z)$ and $x + (y \cdot z) = (x+y) \cdot (x+z)$
8. Complement: $x+x' = 1$ and $x \cdot x' = 0$
9. At least two distinct elements in B

2.2 Key Theorems

Theorem	Expression
Idempotent	$x+x = x$ and $x \cdot x = x$
Null	$x+1 = 1$ and $x \cdot 0 = 0$
Involution	$(x')' = x$
Absorption	$x+(x \cdot y) = x$ and $x \cdot (x+y) = x$
De Morgan	$(x+y)' = x' \cdot y'$ and $(x \cdot y)' = x' + y'$
Consensus	$xy + x'z + yz = xy + x'z$ (yz is the redundant term)

Duality Principle

The dual of a Boolean expression is obtained by replacing every $+$ with $\cdot$, every $\cdot$ with $+$, every 0 with 1, and every 1 with 0. Every valid Boolean theorem also holds for its dual. This lets you derive half the theorems immediately from the other half.

Example: $x + 0 = x$ has dual $x \cdot 1 = x$. $x + x' = 1$ has dual $x \cdot x' = 0$

2.3 Operator Precedence

When parentheses are absent, the evaluation order is:

10. Parentheses (highest priority)
11. NOT (' or $\neg$)
12. AND ($\cdot$)
13. OR ($+$) — lowest priority

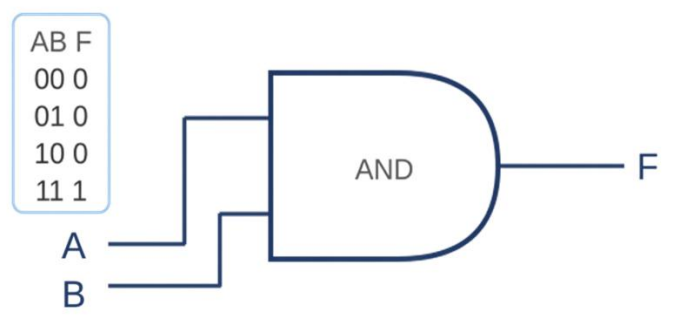
Example: $A + B \cdot C' = A + (B \cdot (C'))$ — NOT is applied to C first, then AND, then OR

2.4 Positive and Negative Logic

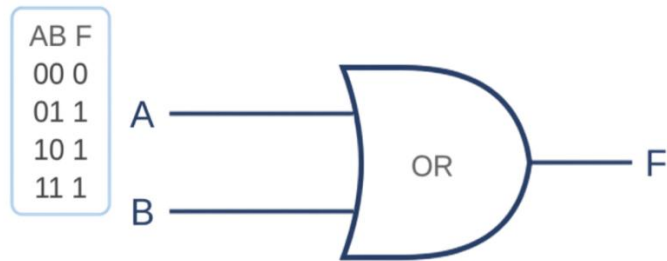
Type	Description
Positive logic (active-high)	High voltage H = logic 1; Low voltage L = logic 0
Negative logic (active-low)	High voltage H = logic 0; Low voltage L = logic 1
Consequence	A positive-logic AND gate becomes a negative-logic OR gate (De Morgan)
Signal notation	Active-low signals often written with overbar or <code>_N</code> suffix (e.g. RESET_N)

2.5 Logic Gate Symbols & Truth Tables

► **AND gate** — $F = A \cdot B$



► **OR gate** — $F = A + B$



► NOT, NAND, NOR gates

NOT $F=A'$



NAND $F=(AB)'$



NOR $F=(A+B)'$



► XOR and XNOR gates

XOR $F=AB'+A'B$



XNOR $F=AB+A'B'$



2.6 Canonical Forms & Standard Forms

$$F = \Sigma (\text{minterms of 1's}) = \Pi (\text{maxterms of 0's})$$

- ◆ Minterm (m_i): AND term where each variable appears exactly once

- ◆ Maxterm (M_j): OR term where each variable appears exactly once
 - ◆ Sum of Products (SOP): sum of AND terms — canonical or simplified
 - ◆ Product of Sums (POS): product of OR terms — canonical or simplified
- $m_j' = M_j$ (complement relationship between minterm and maxterm)

2.7 The 16 Boolean Functions of Two Variables

Function	Name	Comment
$F_0 = 0$	Null	Binary constant 0
$F_1 = xy$	AND	x and y
$F_2 = xy'$	Inhibition	x but not y
$F_3 = x$	Transfer	x
$F_4 = x'y$	Inhibition	y but not x
$F_5 = y$	Transfer	y
$F_6 = xy' + x'y$	XOR	x or y but not both
$F_7 = x + y$	OR	x or y
$F_8 = (x + y)'$	NOR	Not-OR
$F_9 = xy + x'y'$	XNOR	x equals y
$F_{10} = y'$	Complement	Not y
$F_{11} = x + y'$	Implication	If y then x
$F_{12} = x'$	Complement	Not x
$F_{13} = x' + y$	Implication	If x then y
$F_{14} = (xy)'$	NAND	Not-AND
$F_{15} = 1$	Identity	Binary constant 1

★ Chapter 2 Key Points

Duality	Replace $+ \leftrightarrow \cdot$, $0 \leftrightarrow 1$ to get dual theorem for free
Precedence	NOT > AND > OR (parentheses override)
De Morgan	$(AB)' = A' + B'$; $(A + B)' = A'B'$
Canonical	$F = \sum \text{minterms} = \prod \text{maxterms}$

Chapter 3 — Gate-Level Minimization (Karnaugh Maps)

3.1 K-Map Rules

Group Size (4-Variable K-Map)	Literals in Product Term
1 square	4 literals
2 squares	3 literals

4 squares	2 literals
8 squares	1 literal
16 squares	0 literals (constant 1)

- ◆ Cells are arranged in Gray-code order so adjacent cells differ by exactly one variable
- ◆ Groups must be powers of 2: 1, 2, 4, 8, 16
- ◆ Groups may wrap around edges — the map is toroidal (left↔right, top↔bottom)
- ◆ Diagonal groupings are NOT valid — only horizontal/vertical/wrap

4-Variable K-Map Layout (variable order)

Columns labelled AB in Gray-code order: 00 | 01 | 11 | 10. Rows labelled CD: 00 | 01 | 11 | 10. Always use this exact order — NOT binary 00 01 10 11.

AB \ CD	CD=00	CD=01	CD=11	CD=10
AB=00	m ₀	m ₁	m ₃	m ₂
AB=01	m ₄	m ₅	m ₇	m ₆
AB=11	m ₁₂	m ₁₃	m ₁₅	m ₁₄
AB=10	m ₈	m ₉	m ₁₁	m ₁₀

► 3-variable K-map example — $F = C$ (group of 4 → 1 literal)

		00	01	11	10
C=0		0	0	0	0
C=1		1	1	1	1

$F = C$ (group of 4 — 1 literal)

3.2 SOP Minimization (group the 1s)

14. Mark all 1s (and don't-cares X) on the K-map
15. Find all prime implicants — largest possible groupings of 1s
16. Identify essential prime implicants — the only PI covering a given minterm
17. Cover remaining minterms with fewest additional PIs

3.3 POS Minimization (group the 0s)

18. Mark all 0s on the K-map
 19. Group the 0s exactly as you would group 1s for SOP
 20. Each group gives a sum term (OR term) — complement the variables
 21. Multiply all sum terms together to get the POS expression
- Key:** SOP groups 1s → AND terms. POS groups 0s → OR terms. Both give equivalent circuits.

3.4 Prime Implicants & Don't-Cares

- ◆ Prime Implicant (PI): product term from maximum possible adjacent squares
- ◆ Essential PI: the only PI covering a particular minterm — must be included
- ◆ Don't-care (X): input combination that never occurs — treat as 0 or 1 freely

⚠ Never leave an essential PI out of the final expression — it's the only cover for some minterm

3.5 NAND / NOR Implementation

NAND — SOP procedure

22. Simplify to SOP using K-map
23. Draw NAND gate for each product term; inputs = literals of that term
24. Feed all NAND outputs into a final NAND gate (AND-Invert = Invert-OR by De Morgan)
25. Single-literal terms: complement the literal at the NAND input

NOR — POS procedure (dual of NAND)

- ◆ Use POS form. OR-Invert at first level; Invert-OR at second level

3.6 XOR / XNOR Functions

$$x \oplus y = xy' + x'y \quad (\text{odd parity — output 1 when odd number of 1s})$$

$$x \odot y = xy + x'y' = (x \oplus y)' \quad (\text{XNOR — equality, even parity})$$

- ◆ N-variable XOR: output = 1 for odd count of 1s among inputs
- ◆ 2^{n-1} minterms; used for parity generation and checking
- ◆ 3-input odd function: $F = A \oplus B \oplus C$
- ◆ 3-input even function: $F = (A \oplus B \oplus C)' = (A \oplus B \oplus C) \oplus 1$

★ Chapter 3 Key Points

K-map adjacency	Gray-code order — only one variable changes between adjacent cells
POS from K-map	Group the 0s → OR terms → multiply them
Essential PI	Only cover for some minterm — cannot be dropped
Don't-cares	Use freely to enlarge groups; never required to be covered

3.7 Combinational Hazards (Glitches)

A hazard is a momentary glitch on an output caused by unequal propagation delays through different logic paths, even though the final output value is correct. Static hazards are the most common type.

Type	Definition	How to Eliminate
------	------------	------------------

Static-1 Hazard	Output should stay 1 but momentarily glitches to 0 during an input transition	On the K-Map, find two adjacent 1-groups that do NOT share a common implicant. Add a redundant overlapping prime implicant that covers both — this extra gate eliminates the glitch
Static-0 Hazard	Output should stay 0 but momentarily glitches to 1 during an input transition	Work in POS form; add a redundant overlapping sum term on the 0-groups K-Map

3.8 Quine-McCluskey (Tabular) Method

K-Maps become impractical for 5 or more variables because humans cannot visualize higher-dimensional spaces. The Quine-McCluskey (QM) method is the algorithmic alternative that produces the same minimal SOP result mechanically — and can be programmed into a computer.

Step	Action
1. Group by 1-count	List all minterms in binary; sort into groups by number of 1-bits (0 ones, 1 one, 2 ones, ...)
2. Combine adjacent groups	Pairs of minterms that differ in exactly 1 bit position merge; replace differing bit with dash (–). Repeat until no further merging is possible — unchecked terms are prime implicants
3. Prime implicant chart	Build a coverage chart (prime implicants vs minterms). Essential prime implicants (covering a minterm only once) must be included. Select additional PIs to cover remaining minterms with minimum cost
Why use it?	Systematic and computer-programmable — unlike K-Maps it scales to any number of variables (EDA tools use QM internally). Key exam fact: K-Map visual limit is 4–5 variables; QM handles n variables algorithmically

Chapter 4 — Combinational Logic Circuits

4.1 Design & Analysis Procedures

Design Procedure

26. Determine inputs and outputs; assign labels
27. Derive truth table from specification
28. Obtain simplified Boolean functions using K-map
29. Draw logic diagram and verify

Analysis Procedure

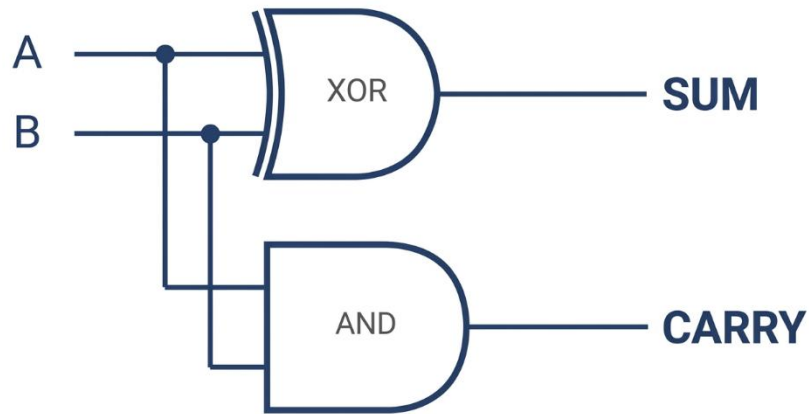
30. Label all gate outputs with symbols
31. Determine Boolean function for each gate output in terms of its inputs
32. Obtain output expressions by substitution from input to output
33. Construct truth table from the final output expressions

4.2 Half Adder

$$\text{Sum} \quad S = A \oplus B$$

$$\text{Carry} \quad C = AB$$

► Half Adder — $S=A \oplus B$, $C=AB$



4.3 Full Adder

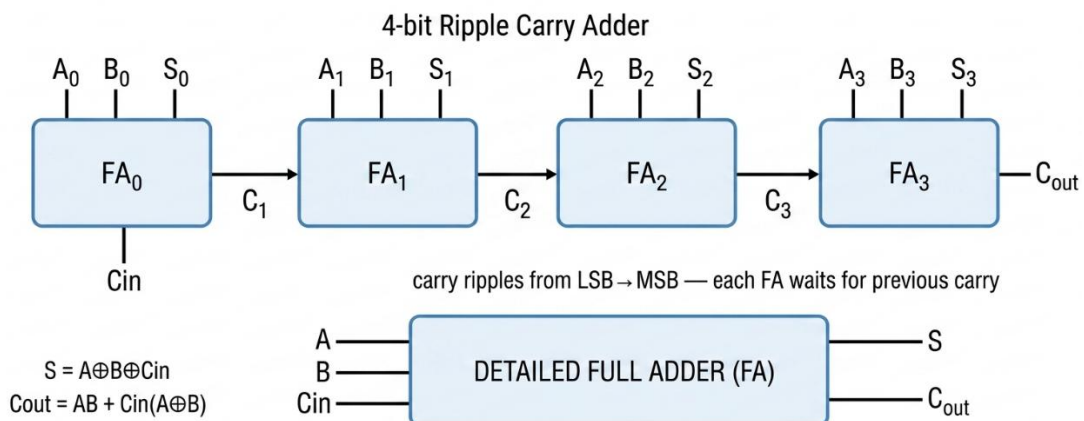
$$\text{Sum} \quad S = A \oplus B \oplus C_{in}$$

$$\text{Carry} \quad C_{out} = AB + C_{in} \cdot (A \oplus B) = AB + AC_{in} + BC_{in}$$

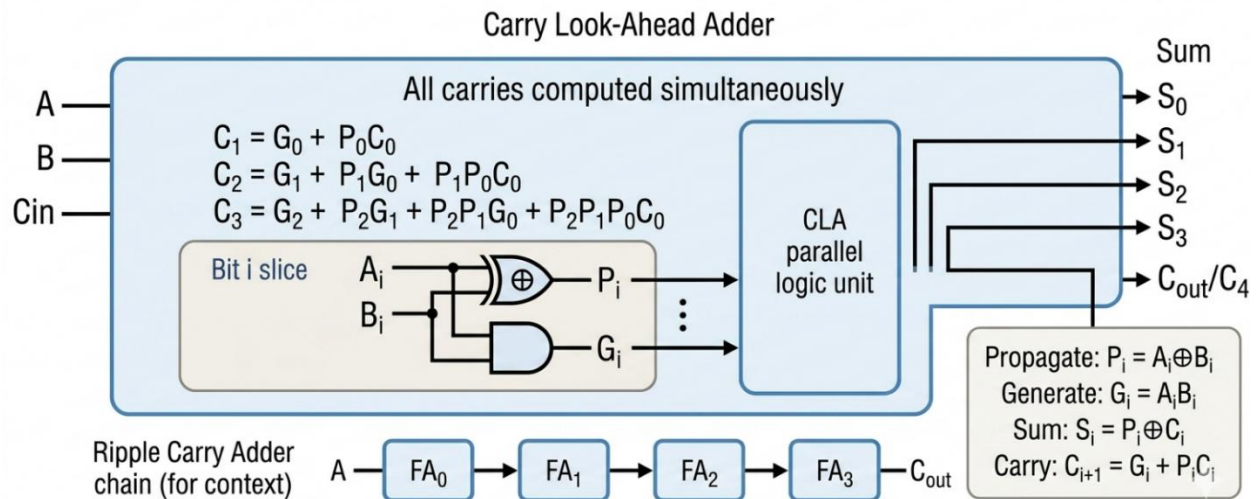
Implementation: Two half adders + one OR gate. The factored form $AB + C_{in}(A \oplus B)$ is used in hardware because $A \oplus B$ is already computed for S .

4.4 Binary Adder & Carry Look-Ahead

► Full Adder and 4-bit Ripple Carry Adder



► Carry Look-Ahead — propagate & generate



Carry Look-Ahead Equations

$$P_i = A_i \oplus B_i \quad (\text{propagate} - \text{carry passes through if } P=1)$$

$$G_i = A_i B_i \quad (\text{generate} - \text{carry created regardless of input carry})$$

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

Deriving C_3 — substituting C_2 to see the recursive pattern:

$$\begin{aligned} C_3 &= G_2 + P_2 \cdot C_2 \\ &= G_2 + P_2 \cdot (G_1 + P_1 \cdot C_1) \quad [\text{substitute } C_2] \\ &= G_2 + P_2 \cdot (G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0)) \quad [\text{substitute } C_1] \\ &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \quad [\text{expanded} - \text{no ripple!}] \end{aligned}$$

Advantage: All carries computed simultaneously in 2-gate delay — no ripple penalty regardless of n

4.5 Binary Adder-Subtractor

XOR gates on B inputs are controlled by mode M. When M=0: XOR passes B unchanged → addition A+B. When M=1: XOR complements B, and C_{in}=M=1 adds 1, giving A + (2's complement of B) = A - B.

Overflow: Detected when carry into MSB ≠ carry out of MSB

4.6 BCD Adder

Add two BCD digits using a 4-bit binary adder. If binary result > 9 or a carry out exists, add 6 (0110) via a second adder to produce a valid BCD output digit.

$$\text{Correction needed when: } K + Z_8 Z_4 + Z_8 Z_2 = 1$$

(K = carry out of first adder; Z₈, Z₄, Z₂ = sum bits)

4.7 Binary Multiplier

Multiplication is achieved by forming partial products (AND of multiplicand with each multiplier bit) and adding them with binary adders.

For J multiplier bits $\times$ K multiplicand bits:

- Need $J \times K$ AND gates (one per partial product bit)
- Need $(K - 1)$ K-bit adders
- Product has $(J + K)$ bits

Example: 2-bit $\times$ 2-bit multiplier: 4 AND gates + one 2-bit adder $\rightarrow$ 4-bit product

4.8 Magnitude Comparator

Equality bit: $X_i = (A_i \oplus B_i)' = A_i B_i + A_i' B_i'$

$A = B$ when $X_3 X_2 X_1 X_0 = 1111$

$A > B = A_3 B_3' + X_3 A_2 B_2' + X_3 X_2 A_1 B_1' + X_3 X_2 X_1 A_0 B_0'$

$A < B = A_3' B_3 + X_3 A_2' B_2 + X_3 X_2 A_1' B_1 + X_3 X_2 X_1 A_0' B_0$

4.9 Decoders & Encoders

Decoder

An $n \times 2^n$ decoder has n input lines and 2^n output lines. Each output represents exactly one minterm. Can implement any Boolean function by ORing the appropriate minterm outputs.

Encoder

Inverse of decoder: 2^n inputs $\rightarrow$ n outputs. Encodes the active input to its binary code.

Octal-to-Binary Encoder (8-to-3) equations

$X = D_4 + D_5 + D_6 + D_7$

$Y = D_2 + D_3 + D_6 + D_7$

$Z = D_1 + D_3 + D_5 + D_7$

Priority Encoder

When multiple inputs are active simultaneously, the highest-priority (highest-index) input is encoded. Outputs include a V (valid) bit = 1 when any input is active; V = 0 means no input is asserted.

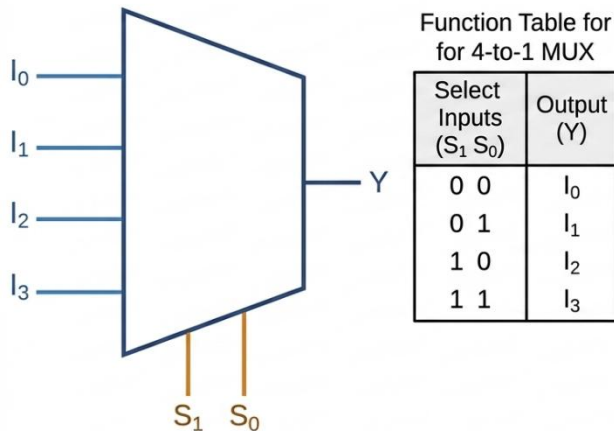
Inputs	Outputs X, Y	Valid V
0 0 0 0	X X (undefined)	0
0 0 0 1	0 0	1
0 0 1 X	0 1	1
0 1 X X	1 0	1
1 X X X	1 1	1

4.10 Multiplexers (MUX)

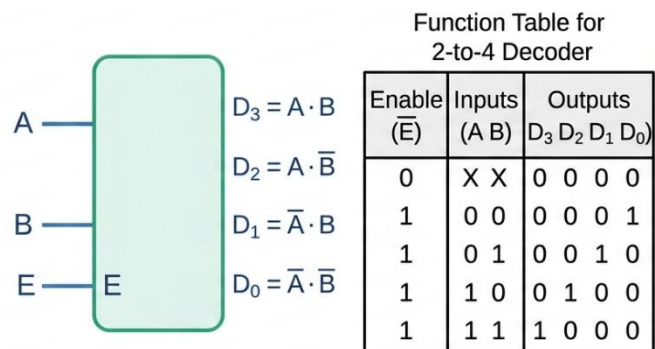
2^n input lines + n selection lines $\rightarrow$ 1 output. Routes selected input to output.

► 4-to-1 MUX and 2-to-4 Decoder

4-to-1 MULTIPLEXER (MUX)



2-to-4 DECODER



- ◆ A 2^n -to-1 MUX can implement any Boolean function of $(n+1)$ variables
- ◆ Connect n variables to select lines; map truth table pairs to data inputs ($0, 1, C_n, C_n'$)

4.11 Three-State Gates

Three output states: logic 0, logic 1, Hi-Z (high impedance = open circuit). When control $C=0$, output is Hi-Z — the gate is effectively disconnected from the bus. Multiple tristate gates can share a single bus if at most one is enabled at a time.

★ Chapter 4 Key Points

Full adder carry	$C_{out} = AB + C_{in}(A \oplus B)$ — use factored form in hardware
Binary multiplier	$J \times K$ AND gates + $(K-1)$ adders $\rightarrow (J+K)$ -bit product
Priority encoder	Highest active input wins; $V=0$ means no input active
MUX as function	2^n MUX implements f of $n+1$ vars using data inputs

Chapter 5 — Verilog HDL

5.1 Modeling Styles

Style	Description
Structural (Gate-Level)	Instantiates primitive gates/modules — most detailed
Dataflow (assign)	Continuous Boolean expressions — synthesisable

Behavioral (always)	Most abstract — if/case/loops
Mixed	Combines any of the above in one module

5.2 Module Structure & Net Types

```
module ModuleName (output y, output [3:0] bus, input A, B, clk);
    wire    w;           // single-driver connection (default)
    wor     w_or;        // wired-OR
    wand    w_and;       // wired-AND
    tri     t;           // tri-state bus
    supply1 vdd;         // logic 1 power supply
    supply0 gnd;         // logic 0 ground
    reg     r;           // holds value (used inside always)
endmodule
```

5.3 Gate Delays & Timescale

```
`timescale 1ns / 10ps    // time unit / precision
and #(2) g1 (out, a, b); // 2ns delay on this gate
not #(1,2) g2 (y, x);    // rise=1ns, fall=2ns delay
```

Purpose: Gate delays enable timing simulation — catching setup/hold violations before fabrication

5.4 Vectors (Buses)

```
output [3:0] y; // y[3]=MSB, y[0]=LSB
wire  [7:0] x; // x[7..0]
wire  [0:3] z; // z[0]=MSB, z[3]=LSB (reversed)
```

5.5 Dataflow Modeling (assign)

Operator	Meaning
& ^ ~	Bitwise AND / OR / XOR / NOT
&& !	Logical AND / OR / NOT
== != === !==	Equality / case equality (includes x,z)
{,}	Concatenation
?:	Conditional (ternary)
<< >> <<< >>>	Logical / arithmetic shifts
**	Exponentiation

5.6 Behavioral Modeling — always block

```
// Combinational — list ALL inputs (or use @(*))
always @(A or B or Cin) begin
    S    = A ^ B ^ Cin;           // blocking = for comb
    Cout = (A&B) | (Cin&(A^B));
```

```
end
```

```
// Sequential – only clock (+ optional async reset)
always @(posedge clk, negedge rst) begin
    if (!rst) Q <= 1'b0;           // async reset
    else      Q <= D;             // non-blocking <= for seq
end
```

Blocking vs Non-Blocking

Type / Rule	Detail
Blocking (=)	Sequential execution; result immediately available → use for combinational
Non-Blocking (<=)	All RHS evaluated first; LHS updated at end of time step → use for registers
Golden rule	= in combinational always; <= in clocked always
Missing sensitivity	Infers unwanted latch — use always @(*) to auto- sense
Missing default	case without default infers latch — always add default:

5.7 Loop Statements

```
repeat (10) @(posedge clk);    // execute 10 times
```

```
forever begin                  // infinite loop
    #5 clk = ~clk;
end
```

```
while (count < 8) begin        // conditional loop
    count = count + 1;
end
```

```
for (i=0; i<8; i=i+1) begin    // counted loop
    memory[i] = 0;
end
```

5.8 Conditional Statements

```
if (sel) out = A; else out = B;
```

```
case (sel)
    2'b00: out = I0;
    2'b01: out = I1;
    2'b10: out = I2;
    2'b11: out = I3;
    default: out = 2'bx; // ALWAYS include default
endcase
```

```

casex (sel) // x and z treated as don't-care
casez (sel) // z only treated as don't-care

```

5.9 User-Defined Primitives (UDP)

UDPs let you define custom gate-level components using a truth table inside the HDL. They are instantiated like standard gates.

```

primitive mux_prim (output Y, input S, I0, I1);
  table
    // S   I0  I1 : Y
    0   0   ?   : 0; // S=0 → pass I0
    0   1   ?   : 1;
    1   ?   0   : 0; // S=1 → pass I1
    1   ?   1   : 1;
  endtable
endprimitive

```

Rules: One output (first in port list); any number of inputs; ? = don't care in table

5.10 Test Bench & Display Tasks

```

module test_tb; // no ports
  reg A, B;
  wire Y;
  dut u1 (.A(A), .B(B), .Y(Y)); // instantiate DUT
  initial begin
    A=0; B=0; #10;
    A=0; B=1; #10;
    $finish;
  end
  initial $monitor("%t A=%b B=%b Y=%b", $time, A, B, Y);
  initial $dumpfile("wave.vcd");
  initial $dumpvars(0, test_tb);
endmodule

```

Blueprint — Sequential FSM Testbench (UUT = Unit Under Test):

```

module tb_mealy; // no port list — testbenches have none
  // 1. Drive inputs with reg (procedural blocks can assign reg)
  reg clk, rst, x_in;
  // 2. Capture outputs with wire (driven by the UUT)
  wire y_out;
  // 3. Instantiate the design under test
  mealy_fsm UUT (.clk(clk), .rst(rst), .x(x_in), .y(y_out));
  // 4. Free-running clock — toggles every 5 time units (period = 10)
  initial clk = 0;
  always #5 clk = ~clk;
  // 5. Stimulus — drives inputs, waits, checks outputs
  initial begin
    rst=1; x_in=0; #10; // apply reset
    rst=0; #10; // release reset
    x_in=1; #20; // drive stimulus
    x_in=0; #10;
    $display("y_out=%b at t=%0t", y_out, $time);
    $finish; // end simulation
  end
endmodule

```

```
end
endmodule
```

Format Specifiers for \$display / \$monitor / \$write

Specifier	Output format
%b or %B	Binary
%d or %D	Decimal
%h or %H	Hexadecimal
%o or %O	Octal
%t	Time value
%s	String

5.11 Verilog Operator Precedence (High → Low)

Priority	Operators
Highest	+ - ! ~ & ^ ^~ (unary)
	* / % then ** (power)
	+ - (binary)
	<< >> <<< >>>
	< <= > >=
	== != === !==
	& then ^ then (binary reduction)
	&&
Lowest	?: (conditional) { } (concatenation)

★ Chapter 5 Key Points

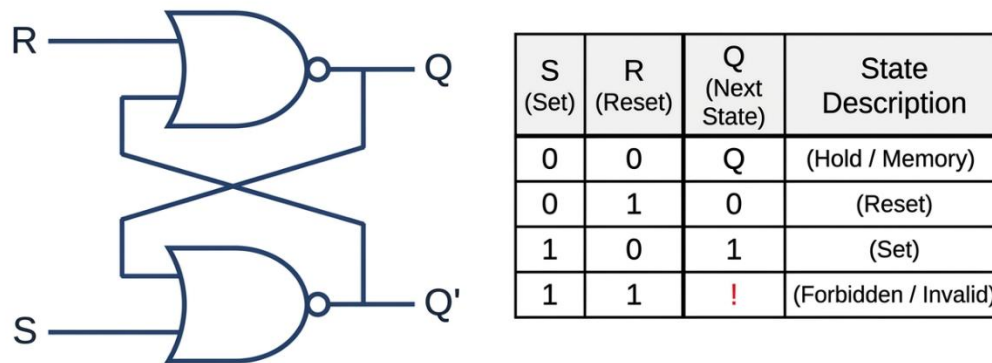
Sensitivity list	Comb: @(*) or list all; Seq: @(posedge clk) only
Assignment	= for combinational; <= for sequential (registers)
UDP	Custom truth-table primitives; one output; ? = don't care
Loops	repeat/forever/while/for — all inside initial or always

Chapter 6 — Sequential Logic & Storage Elements

6.1 SR Latch (NOR-gate)

► SR NOR Latch with cross-coupled feedback

SR Latch: Circuit Diagram and Truth Table



S, R	Output Q
S=0, R=0	No change — latch holds current state (memory)
S=0, R=1	Q = 0 (Reset)
S=1, R=0	Q = 1 (Set)
S=1, R=1	Forbidden — both Q and Q' forced to 0

NAND SR Latch (active-low inputs $\bar{S}\bar{R}$)

$\bar{S}$, $\bar{R}$ inputs	Output Q
$\bar{S}=1$, $\bar{R}=1$	No change (memory)
$\bar{S}=1$, $\bar{R}=0$	Q = 0 (Reset — active low)
$\bar{S}=0$, $\bar{R}=1$	Q = 1 (Set — active low)
$\bar{S}=0$, $\bar{R}=0$	Forbidden — both outputs forced to 1

6.2 SR Latch with Control Input (Gated SR)

Adding an Enable (E_n) input gates the SR latch. When $E_n=0$ the latch is isolated regardless of S and R. When $E_n=1$ the latch operates normally.

E_n	S	R	Next Q
0	X	X	No change
1	0	0	No change

1	0	1	Q = 0 (Reset)
1	1	0	Q = 1 (Set)
1	1	1	Forbidden

6.3 D Latch (Gated)

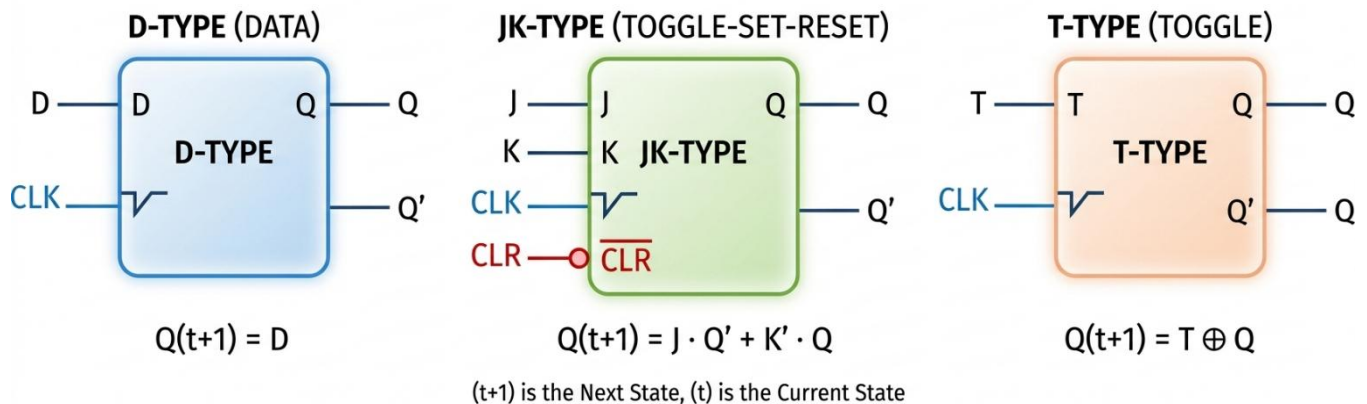
The D latch eliminates the forbidden state of the SR latch by forcing $S = D$ and $R = D'$. When $En=1$: transparent — Q follows D. When $En=0$: Q holds last value.

$$Q(t+1) = D \quad \text{when } En = 1$$

6.4 Edge-Triggered Flip-Flops

► D, JK and T Flip-Flops with characteristic equations

FLIP-FLOP OVERVIEW AND CHARACTERISTIC EQUATIONS



Characteristic Tables & Equations

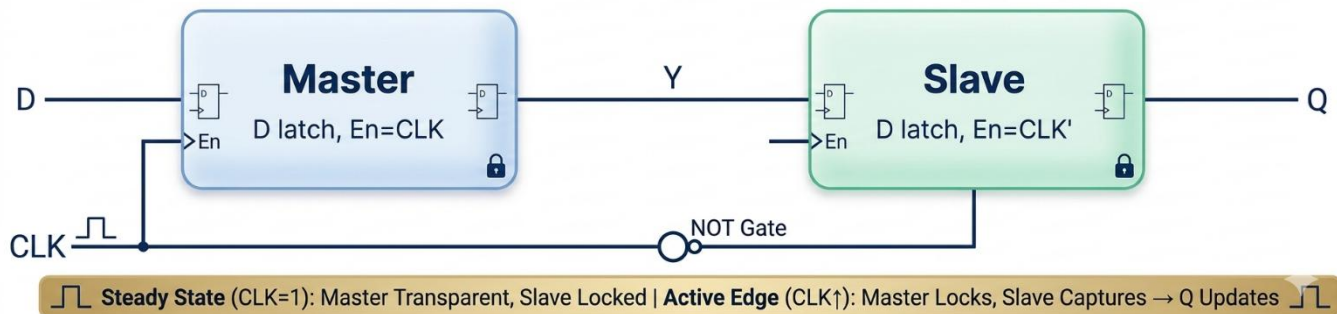
D-FF		JK-FF	
D	Q(t+1)	JK	Q(t+1)
0	0	00	Q(t) no change
1	1	01	0 reset
		10	1 set
		11	Q'(t) toggle

$D\text{-FF} : Q(t+1) = D$
 $JK\text{-FF} : Q(t+1) = JQ' + K'Q$
 $T\text{-FF} : Q(t+1) = T \oplus Q = TQ' + T'Q$

6.5 Master-Slave D Flip-Flop

► Master-Slave D Flip-Flop — edge triggering

Master-Slave D Flip-Flop — Edge Triggering Mechanism



Two D latches in series with inverted enables. Master is transparent on CLK=1 (captures D). Slave captures master's output on CLK↓. Q only updates on the falling clock edge, eliminating transparency and 1s-catching.

6.6 Asynchronous vs Synchronous Reset

Type	Description
Synchronous reset	Reset acts only on the active clock edge — if (1rst) $Q \leq 0$ inside clocked always
Asynchronous reset	Reset overrides clock immediately — add negedge rst to sensitivity list
Verilog rule	Asynchronous events first in sensitivity list; last else = clock operation
Preference	Async reset for initialisation; sync reset for functional resets in designs

★ Chapter 6 Key Points

SR latch states	00=hold, 01=reset, 10=set, 11=FORBIDDEN
Gated SR	En=0 isolates latch; En=1 normal operation
D latch	Transparent when En=1; stores when En=0
D/JK/T equations	$D=D$; $JK=JQ'+K'Q$; $T=T \oplus Q$

6.7 Setup & Hold Times — Timing Constraints

Timing violations are the most common cause of sequential circuit failure. Every flip-flop imposes a minimum window around the active clock edge during which the data input must be stable.

Parameter	Definition	Violation Effect
-----------	------------	------------------

Setup Time (t_{su})	Minimum time D must be stable BEFORE the active clock edge	FF enters metastability — output unpredictable (0 or 1)
Hold Time (t_h)	Minimum time D must remain stable AFTER the active clock edge	FF captures wrong data — corrupted output even though value settled before the edge
Max Clock Freq	$f_{max} = 1 / (t_{cq} + t_{comb} + t_{su})$ where t_{cq} = FF clock-to-Q delay	

Chapter 7 — Sequential Circuit Analysis & Design

7.1 Analysis Procedure

Given a clocked sequential circuit, the analysis procedure produces the state table and state diagram:

34. Write FF input equations from the circuit (e.g. $J = Ax$, $K = B + x'$)
35. Write state equations using characteristic equations: substitute FF inputs into $Q(t+1) = f(J,K,Q)$
36. Build the state table: rows = all (present-state, input) combinations; columns = next state, output
37. Draw the state diagram: circles = states, directed arcs = transitions labelled input/output

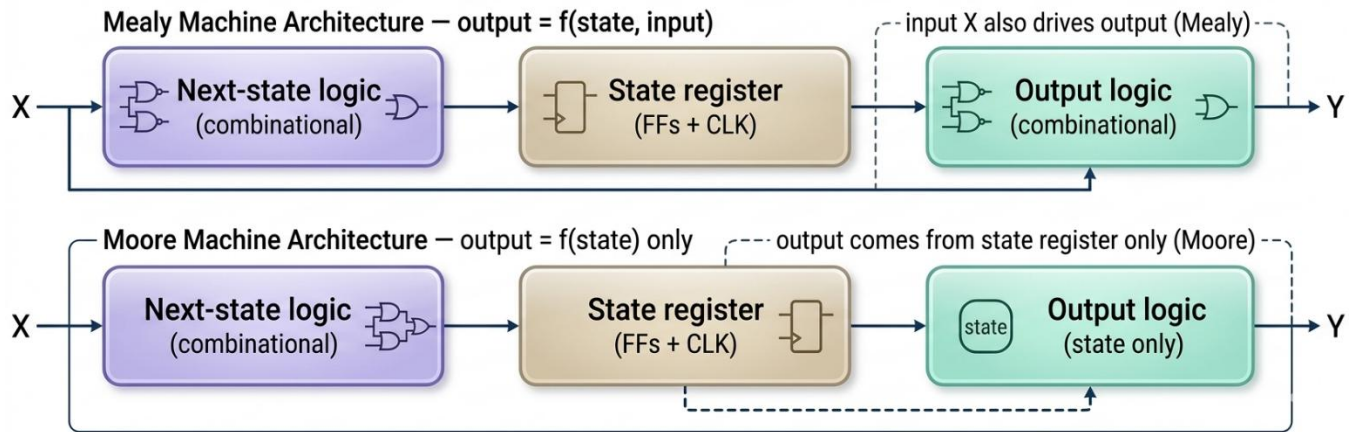
State Table Template

Present State	Input X	Next State	Output Y
Present State	Input X	Next State	Output Y
A	0	A	0
A	1	B	0
B	0	C	0
B	1	A	1

7.2 Mealy vs Moore Machines

► Mealy vs Moore FSM architecture

COMPARISON OF MEALY VS MOORE FSM ARCHITECTURES



Machine	Key Characteristic
Mealy	Output = $f(\text{state}, \text{input})$ — fewer states needed; output labelled on arc (input/output)
Moore	Output = $f(\text{state})$ only — more states; output labelled inside state circle
Mealy advantage	Responds to input changes within same clock cycle
Moore advantage	Outputs are glitch-free — only change after a clock edge

7.3 Design Procedure (7 Steps)

38. Derive state diagram from word description
39. Reduce number of states — merge equivalent states
40. Assign binary codes to states (binary / Gray code / one-hot)
41. Obtain the binary-coded state table
42. Choose flip-flop type: D, JK, or T
43. Derive FF input equations + output equations using K-maps and excitation tables
44. Draw the logic diagram and verify correctness

7.4 Excitation Tables

JK Flip-Flop

Transition $Q(t) \rightarrow Q(t+1)$	Required J, K
$Q(t)=0 \rightarrow Q(t+1)=0$	$J=0, K=X$ (K doesn't matter — Q already 0)
$Q(t)=0 \rightarrow Q(t+1)=1$	$J=1, K=X$
$Q(t)=1 \rightarrow Q(t+1)=0$	$J=X, K=1$
$Q(t)=1 \rightarrow Q(t+1)=1$	$J=X, K=0$ (J doesn't matter — Q already 1)

T Flip-Flop

Transition	Required T
$0 \rightarrow 0$	$T = 0$ (no toggle needed)
$0 \rightarrow 1$	$T = 1$ (must toggle)
$1 \rightarrow 0$	$T = 1$ (must toggle)
$1 \rightarrow 1$	$T = 0$ (no toggle needed)

7.5 State Reduction & Assignment

Two states are equivalent if they produce identical outputs for every input AND transition to equivalent next states. Equivalent states can be merged — this reduces the number of FFs and gates.

Method 1 — Row Elimination (quick, manual)

Scan the state table for two rows with identical outputs AND identical next-state columns. If found, those two states are equivalent — replace all references to one with the other and delete its row. Repeat until no further merges are possible.

Method 2 — Implication Table (systematic, exam-safe)

#	Step
1	Draw a triangular grid with states $S_1 \dots S_n$ on both axes. Each cell (S_i, S_j) represents the pair to be tested
2	Cross out (X) any cell where the two states produce different outputs for any input — these pairs are immediately non-equivalent
3	For remaining cells, write in the implied pairs — the next-state pairs that must also be equivalent. E.g., if (S_1, S_2) requires (S_3, S_4) to hold, write " S_3S_4 " in that cell
4	Iterate: if an implied pair was crossed out, cross out the current cell too. Repeat until no more crosses propagate. Remaining uncrossed cells are equivalent pairs — merge those states

Method	Trade-off
Binary encoding	Sequential binary codes — most compact: $\log_2 n$ bits for n states. Best for ASIC/counter designs where minimizing FF count is critical.
Gray code	Adjacent states differ by 1 bit — fewer transitions, less glitching, lower power consumption. Use when power or EMI is a concern, or for counter outputs driving analog circuits.

One-hot

One FF per state — simple decode, no combinational decoding logic needed. Ideal for high-speed FPGA designs where FFs are abundant and fast next-state logic is the priority.

7.6 Self-Correcting Machines

A self-correcting machine is one where every unused (invalid) state transitions to a valid state within a finite number of clock cycles. This is critical for robustness — if noise or power-up puts the circuit into an invalid state, it recovers automatically.

- ◆ During design: assign don't-cares to unused states in the K-map
- ◆ After simplification: check all unused states — verify each transitions to a valid state
- ◆ If an unused state loops back to itself → NOT self-correcting; redesign

★ **Exam tip:** Always verify unused state transitions after minimisation — this is a common exam question

★ Chapter 7 Key Points

Analysis	Circuit → FF equations → state table → state diagram
Mealy vs Moore	Mealy: output on arc; Moore: output in state
Self-correcting	Every invalid state must reach a valid state
JK excitation	$0 \rightarrow 0: J=0, Kx$; $0 \rightarrow 1: J=1, Kx$; $1 \rightarrow 0: Jx, K=1$; $1 \rightarrow 1: Jx, K=0$

Chapter 8 — Registers & Counters

8.1 Registers

A register is a group of n flip-flops sharing a common clock, capable of storing n bits. A register with parallel load captures external data on the rising clock edge when Load=1; when Load=0 Q is fed back to its own D input (hold).

8.2 Shift Registers & Serial Transfer

Data shifts one position per clock pulse. A 4-bit shift register moves data from $Q_3 \rightarrow Q_2 \rightarrow Q_1 \rightarrow Q_0$ (shift right) or $Q_0 \rightarrow Q_1 \rightarrow Q_2 \rightarrow Q_3$ (shift left). Serial transfer between two registers copies all bits in n clock pulses using a single wire.

Universal Shift Register — 4 operating modes

Mode S_1S_0	Operation
$S_1=0, S_0=0$	Hold — all FFs keep current state
$S_1=0, S_0=1$	Shift right — data $Q_3 \rightarrow Q_2 \rightarrow Q_1 \rightarrow Q_0$; SI_R enters Q_3
$S_1=1, S_0=0$	Shift left — data $Q_0 \rightarrow Q_1 \rightarrow Q_2 \rightarrow Q_3$; SI_L enters Q_0
$S_1=1, S_0=1$	Parallel load — all 4 bits loaded from $I_3I_2I_1I_0$ simultaneously

8.3 Serial Adder

A serial adder adds two n-bit numbers one bit at a time per clock cycle. It requires much less hardware than a parallel adder but takes n clock cycles.

Components: Shift Register A (augend) + Shift Register B (addend) + Full Addder + carry flip-flop (D-FF).

Serial Adder State Table

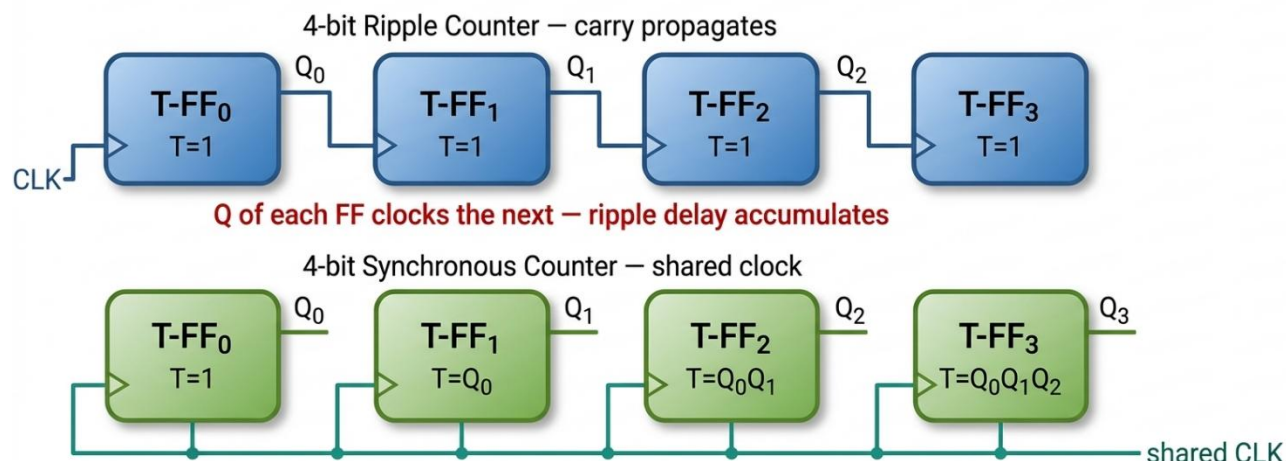
Q	x (A)	y (B)	Next Q'	Sum S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$J_C = xy, \quad K_C = (x'y)' = x+y', \quad S = x \oplus y \oplus Q$$

8.4 Counters

► 4-bit Ripple Counter vs 4-bit Synchronous Counter

Comparison: 4-Bit Ripple vs. 4-Bit Synchronous Counter



Ripple (Asynchronous) Binary Counter

- ◆ Q of each T-FF clocks the next → carry ripples LSB to MSB
- ◆ Propagation delay accumulates — all outputs do not change simultaneously

Synchronous Binary Counter

- ◆ All FFs share the same clock — no ripple delay
- $T_0 = 1$ $T_1 = Q_0$ $T_2 = Q_1Q_0$ $T_3 = Q_2Q_1Q_0$

8.5 BCD Ripple Counter — State Sequence

The BCD counter counts in the standard 8421 BCD sequence and resets after 9 (1001):

0000 → 0001 → 0010 → 0011 → 0100 → 0101 → 0110 → 0111 → 1000 → 1001 → 0000

The feedback logic forces Q_1 to reset when $Q_3=1$ AND $Q_1=1$ (i.e. count = 10 = 1010). Chain multiple BCD counters for multi-digit decimal counting (ones → tens → hundreds...).

8.6 Special Counters

Type	Description
Up-Down counter	Control UP=1 counts up; DOWN=1 counts down. TUP: $T_i = Q_0Q_1...Q_{i-1}$; TDOWN: $T_i = Q_0'Q_1'...Q_{i-1}'$
Ring counter	n FFs → n states; Q of last FF feeds back to D of first. Initial = 1000...0. Generates n non-overlapping timing pulses.
Johnson counter	Q' of last FF feeds back → $2n$ states; self-decoding — each state decoded with one 2-input AND gate.
Decode counter	Ring counter + decoder: 2^n timing signals use n -FF ring + n -to- 2^n decoder

8.7 Counter with Parallel Load

Control Signals	Function
Clear=0	Force Q=0 asynchronously — overrides all other inputs
Clear=1, Load=1	Load parallel input on next $\uparrow$ CLK
Clear=1, Load=0, Count=1	Count up on next $\uparrow$ CLK
Clear=1, Load=0, Count=0	No change

★ Chapter 8 Key Points

Serial adder	n-clock cycles; FA + carry FF + two shift registers
BCD counter	0000 $\rightarrow$ 1001 $\rightarrow$ 0000; use Q_3Q_1 feedback to force reset at 10
Johnson counter	n FFs $\rightarrow$ 2n states; Q' feedback; 2-input AND decode
USR modes	00=hold; 01=shift right; 10=shift left; 11=parallel load

Chapter 9 — Verilog HDL for Sequential Circuits

9.1 initial vs always — Behavioral Modeling Patterns

Block	Purpose
initial block	Executes once at time 0; used for testbench stimulus and variable initialisation
always block	Repeats forever; used for circuit description (clocks, registers, combinational logic)

Clock Generation with initial + always

```
initial clock = 1'b0;           // set at time 0
always #5 clock = ~clock;       // toggle every 5ns  $\rightarrow$  10ns period
```

Single-Pass Stimulus (initial only)

```
initial begin
    clock = 1'b0;
    repeat(30) #10 clock = ~clock; // 30 transitions then stop
end
```

9.2 HDL Models for Storage Elements

D Latch

```
always @(enable or D) if (enable) Q <= D;
```

D Flip-Flop — positive edge

```
always @(posedge clk) Q <= D;
```


D FF with Async Reset

```
always @(posedge clk, negedge rst)
  if (!rst) Q <= 1'b0;
  else    Q <= D;
```

JK Flip-Flop

```
always @(posedge clk)
  case ({J,K})
    2'b00: Q <= Q;      // hold
    2'b01: Q <= 1'b0;   // reset
    2'b10: Q <= 1'b1;   // set
    2'b11: Q <= ~Q;     // toggle
  endcase
```

T Flip-Flop

```
always @(posedge clk) Q <= Q ^ T;
```

9.3 Clock Correctness Rules

- 45. if-else-if structure → if-conditions = asynchronous events (rst, set)
- 46. Last else → synchronous (clocked) operation
- 47. Asynchronous events listed first in sensitivity list
- 48. Never include D or combinational signals in a clocked always sensitivity list

9.4 Nonblocking Assignments & Inter-Assignment Delay

The nonblocking assignment with a delay `Q <= #2 ~Q` schedules the update 2 time units in the future. This models propagation delay but can cause unexpected simulation behaviour if overused. Ripple counters in Verilog simulation use this pattern.

```
// Swap A and B safely using non-blocking (no temp variable needed)
always @(posedge clk) begin
  A <= B;    // RHS evaluated simultaneously
  B <= A;    // correct swap - A not yet updated
end
```

9.5 parameter Declaration — State Encoding

```
parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
reg [1:0] state, next_state;
```

Purpose: Named constants make state assignments readable and easy to change. Replace 2'b00 with the parameter name everywhere.

9.6 FSM Coding — Two-Always Style

Mealy Machine HDL Pattern

⚠ Critical Rule — Blocking vs Non-Blocking in FSMs:

Always 1 (sequential / state register)	→ use	<=	(non-blocking)
Always 2 (combinational / next-state)	→ use	=	(blocking)

Mixing these causes synthesis-simulation mismatches — the simulator may show correct behavior while the synthesized circuit is broken.

```
// Always 1: state register (sequential)
always @(posedge clk, negedge rst)
    if (!rst) state <= S0;
    else      state <= next_state;

// Always 2: next-state + output (combinational)
always @(state, x_in) begin
    case (state)
        S0: if (x_in) begin next_state=S1; y_out=1; end
            else      begin next_state=S0; y_out=0; end
        S1: ...
        default: next_state = S0; // self-correcting!
    endcase
end
```

Moore Machine HDL Pattern

```
// Always 1: same state register as Mealy
// Always 2: next-state (combinational)
always @(state, x_in)
    case (state)
        S0: next_state = x_in ? S1 : S0;
        S1: next_state = x_in ? S1 : S2;
        default: next_state = S0;
    endcase

// Always 3: output (function of state ONLY — Moore)
always @(state)
    case (state)
        S0: y_out = 0;
        S1: y_out = 1;
        default: y_out = 0;
    endcase
```

9.7 RTL Design Methodology

- ◆ Separate into datapath (ALU, registers, MUXes) and controller (FSM)
- ◆ Controller generates control signals; datapath executes register operations
- ◆ ASM chart: graphical state machine with decision diamond and output rectangle boxes
- ◆ ASMD chart: ASM chart annotated with concurrent datapath register operations

9.8 HDL Design Flow

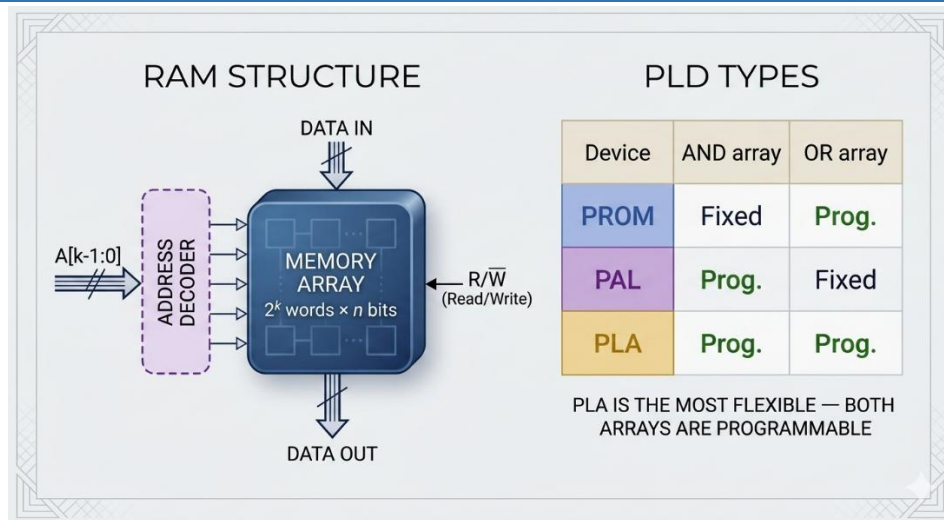
49. Specification
50. RTL Verilog coding (structural, dataflow, or behavioral)
51. Functional simulation — test bench verifies logic correctness
52. Logic synthesis — HDL → gate netlist
53. Timing simulation — gate delays, setup/hold checks
54. Place & Route — physical layout, wire routing

★ Chapter 9 Key Points

initial vs always	initial: once at $t=0$ (stimulus); always: forever (circuit)
parameter	Named constants for state encoding — use everywhere
Mealy HDL	2 always blocks; Moore HDL: 3 always blocks
Self-correcting FSM	Always add default: <code>next_state=S0</code> in case statement

Chapter 10 — Memory & Programmable Logic

► RAM structure and PLD types (PROM / PAL / PLA)



10.1 RAM — Structure & Operation

A memory unit of 2^k words $\times$ n bits. Has k address lines, n data I/O lines, and read/write control. $2^k \geq m$ where m is the number of words needed.

Term	Definition
Access Time	Time to select a word and read it out
Cycle Time	Total time to complete a write operation ($\geq$ access time)
SRAM	SR latches store bits — fast, stable, volatile, no refresh needed
DRAM	Capacitors store bits — must be refreshed every few ms; denser, cheaper
Non-volatile	ROM, Flash, Magnetic disk — data survives power-off
Volatile	SRAM, DRAM — data lost on power-off

Write & Read Procedures

Write:

56. Apply binary address of target word to address lines
57. Apply data bits to data input lines
58. Activate the Write control signal

Read:

59. Apply binary address of target word to address lines
60. Activate the Read control signal — data appears on output lines

10.2 Memory Cell (SRAM)

A single SRAM cell stores one bit using a cross-coupled SR latch plus access transistors. The cell is selected by the word line (row address). Data is read/written via the bit lines (column).

Signal	Function
Word line (select)	Row address decoder output — selects the entire row of cells
Bit line	Column data path — carries data in (write) or out (read)
Select=0	Cell isolated from bit lines — holds its state indefinitely
Select=1, R/W=1	Read: cell drives the bit line
Select=1, R/W=0	Write: bit line forces the cell to new state

10.3 Coincident Decoding (2D Decoding)

For large memories, a single n -to- 2^n decoder becomes impractical. Coincident decoding splits the address into two halves: a row address (decoded by row decoder) and a column address (decoded by column decoder). A cell is selected only when its row AND column are both active.

Example: 1024-word memory = 32×32 array

Row decoder: 5-bit → 1-of-32 rows (5×32 decoder)

Column decoder: 5-bit → 1-of-32 cols (5×32 decoder)

Total: 2 decoders of 32 outputs each vs 1 decoder of 1024 outputs

10.4 DRAM & Address Multiplexing

To reduce pin count in DRAM chips, the row and column addresses share the same address bus. Two strobe signals control which is loaded:

Signal / Concept	Description
RAS (Row Address Strobe)	Falling edge latches the row address into the row address register
CAS (Column Address Strobe)	Falling edge latches the column address — then selects the column
Advantage	Halves the number of address pins needed (e.g. 16 pins serve a $256 \times 256 = 64K$ array)

Refresh

Each row must be read (refreshed) every few ms
— done by a refresh controller

10.5 ROM & Programmable Devices

ROM stores $2^k \times n$ bits permanently. Non-volatile. The AND array is a fixed decoder — all 2^k minterms are always present. The OR array selects which minterms contribute to each output.

32×8 ROM — Implementation

5 address inputs (I_4 – I_0) drive a 5-to-32 decoder producing 32 minterms. Each of the 8 outputs (A_7 – A_0) ORs the minterms assigned to it through intact fuses.

Fuse intact → minterm contributes to output (logic connection)

Fuse blown → minterm disconnected from output

PLD Comparison

Device	Structure	Trade-off
PROM	Fixed AND (decoder) + Programmable OR	Implements any SOP; large for sparse functions
PAL	Programmable AND + Fixed OR	Smaller/faster than PROM; limited OR terms per output
PLA	Programmable AND + Programmable OR	Most flexible; product terms shared across outputs

PLA Programming Convention

- ◆ X (fuse intact) → variable appears in the product term
- ◆ + (fuse blown) → variable absent from the product term
- ◆ XOR bit = 1 → complement the output; XOR bit = 0 → true output

★ Chapter 10 Key Points

SRAM cell	SR latch + word line (row select) + bit lines (data)
Coincident decode	Split address → row + column decoder; cell = row AND column
DRAM RAS/CAS	Shared address bus; RAS loads row, CAS loads column
ROM fuse	Intact = connected; Blown = disconnected

10.6 FPGA Architecture — LUTs & CLBs

Modern FPGAs do not implement logic with physical AND/OR gates. Instead, they use Look-Up Tables (LUTs) — tiny SRAM memories where the input combination is the address and the stored bit is the output. A 4-input LUT can implement any Boolean function of 4 variables by pre-loading the 16-entry truth table into its 16 SRAM cells.

Component	Role
LUT (Look-Up Table)	Tiny SRAM with n address bits and 2^n data bits. Input signals are the address; the pre-programmed bit at that address is the output. Implements any combinational function of n variables — no AND/OR gates needed

D Flip-Flop (FF)	Paired with each LUT to optionally register its output, enabling sequential logic. Controlled by a clock enable and synchronous/asynchronous reset
MUX (Output Select)	Selects whether the CLB output comes from the LUT directly (combinational) or from the FF (registered/sequential)
CLB (Configurable Logic Block)	The fundamental repeating unit of an FPGA: LUT + FF + MUX grouped together. Thousands to millions of CLBs tile the FPGA die, interconnected by a programmable routing fabric. Compare to PLA/ROM: LUT = truth-table SRAM; PLA = fixed AND-OR array; ROM = full decoder

Quick Reference — Key Formulas & Facts

Formula / Concept	Expression / Rule	Notes
2's Complement	Invert all bits, then add 1	Negate; range: -2^{n-1} to $+2^{n-1}-1$
Overflow	$(+V)+(+V)=-V$ or $(-V)+(-V)=+V$	2's complement overflow rule
Half Adder	$S = A \oplus B$, $C = AB$	1-bit add, no carry-in
Full Adder	$S = A \oplus B \oplus C_{in}$, $C = AB + C_{in}(A \oplus B)$	1-bit add with carry-in
CLA carry	$C_{i+1} = G_i + P_i C_i$	Propagate/generate; 2-gate delay
Binary multiplier	$J \times K$ ANDs + $(K-1)$ adders $\rightarrow (J+K)$ bits	Partial product method
SOP canonical	$F = \Sigma \text{ minterms (1s)}$	Sum of minterms
POS canonical	$F = \Pi \text{ maxterms (0s)}$	Product of maxterms
De Morgan	$(A+B)' = A'B'$; $(AB)' = A'+B'$	Complement rule
Duality	Replace $+ \leftrightarrow \cdot$, $0 \leftrightarrow 1$	Derives dual theorem
Operator precedence	NOT > AND > OR	Parentheses override
D-FF	$Q(t+1) = D$	Captures D on clock edge
JK-FF	$Q(t+1) = JQ' + K'Q$	J=set, K=reset, J=K=1 toggle
T-FF	$Q(t+1) = T \oplus Q$	T=0: hold; T=1: toggle
XOR	$A \oplus B = AB' + A'B$	Odd parity / inequality
XNOR	$(A \oplus B)' = AB + A'B'$	Equality / even parity
Serial adder	n clock cycles; FA + carry FF + 2 shift regs	Serial addition hardware
Ring counter	n FFs $\rightarrow$ n states	Q of last FF feeds back
Johnson counter	n FFs $\rightarrow$ 2n states	Q' of last FF feeds back
BCD counter	0000 $\rightarrow$ 1001 $\rightarrow$ 0000	$Q_3 Q_1$ feedback forces reset at 10

MUX as function	$2^n \text{ MUX} \rightarrow f \text{ of } n+1 \text{ vars}$	Map pairs of rows to data inputs
Octal encoder	$X = D_4 + D_5 + D_6 + D_7$	OR groups for each output bit
RAM capacity	$2^k \text{ words (k = address bits)}$	Memory size
DRAM refresh	RAS latches row; CAS latches col	Shared address bus, halves pin count
Coincident decode	Row decoder $\times$ column decoder	2D addressing for large arrays
K-map POS	Group 0s $\rightarrow$ OR terms $\rightarrow$ multiply	Dual of SOP grouping
Self-correcting	Every invalid state $\rightarrow$ valid state	Add default in case; check unused states
UDP	primitive...table...endtable	Custom truth-table gate primitive
parameter	Named constant for state codes	Improves readability and maintainability
FSM Verilog	Mealy: 2 always; Moore: 3 always	Output placement differs